

Research Documentation - End-to-End Testing with Maestro for SecureShift Guard App

Table of Contents

Overview	2
What is Maestro?	2
Why was Maestro considered?	3
Comparison Between Current Testing and Maestro	5
Research Findings	5
1. Large Application Surface Area	5
2. Time Required for Setup	6
3. Team Requirements	7
4. Device and Environment Complexity	7
Proposed Future Architecture	8
Example Maestro Flows for SecureShift	8
Login flow	8
Invalid Login Flow	8
Navigation Flow	9
Benefits of Future Maestro Implementation	9
Limitations and Risks	9
Flaky Tests	9
Maintenance Overhead	9
Additional Build Complexity	9
Time Constraints	10
Current Recommendation	10
Continue Using Jest + React Native Testing Library	10
Prepare Documentation for Future Cohorts	10
Suggested Future Implementation Plan	11
Phase 1 - Research and Setup	11
Phase 2 - Core Authentication Flows	11
Phase 3 - Main Application Flows	11
Phase 4 - CI/CD and Maintenance	12
Conclusion	12

Overview

This document was prepared as research into implementing end-to-end (E2E) testing for the SecureShift Guard App using Maestro.

The purpose of this research is to explore whether Maestro could be used in the project for automated UI and workflow testing in the future, especially for later cohorts or future development phases.

After researching and experimenting with the current project structure, it was found that full implementation of Maestro across the Guard App would require significant setup time, multiple contributors, and continuous maintenance. Due to the limited timeframe remaining in Trimester 1, a complete implementation would not be feasible at this stage.

However, this document outlines:

- What Maestro is
- Why it was considered
- Benefits and limitations
- How Maestro could be implemented in SecureShift
- Estimated workload and team requirements
- Example test flows
- Suggested future implementation plan

What is Maestro?

Maestro is a mobile UI testing framework used for automated end-to-end testing on Android and iOS applications.

Unlike unit testing frameworks such as Jest and React Native Testing Library, Maestro tests the application from the user perspective. It interacts with the actual application UI by:

- Pressing buttons
- Typing into inputs
- Navigating between screens
- Checking visible text
- Testing real user flows

Maestro tests are written in YAML files, which makes them simpler to read compared to some other mobile testing frameworks.

Example:

```
appId: com.secureshiftguardapp
---
- launchApp
- tapOn: "Email"
- inputText: "guard@test.com"
- tapOn: "Password"
- inputText: "password123"
- tapOn: "Login"
- assertVisible: "Home"
```

Why was Maestro considered?

The Guard App currently uses Jest and React Native Testing Library for component and validation testing.

These tests are useful for:

- Rendering checks
- Validation checks
- Navigation checks
- Unit-level behaviour
- Input handling
- Basic interaction testing

This testing setup works well for verifying that individual screens and components behave correctly in isolation.

However, while researching the project testing structure, it became clear that the current setup does not fully simulate real user behaviour across the entire application.

For example, current tests may confirm:

- A button exists
- Validation appears correctly
- Navigation functions are called

But they do not fully test:

- Whether a user can successfully move through multiple screens
- Whether workflows break after UI updates
- Whether device-level interactions behave correctly
- Whether asynchronous flows work consistently on real devices or emulators

Maestro was considered because it allows testing from the perspective of a real user interacting with the app.

Instead of testing isolated components, Maestro can automate complete workflows such as:

- Launching the application
- Entering login credentials
- Navigating between screens
- Submitting forms
- Checking visible UI states
- Verifying successful navigation
- Testing failure scenarios

Maestro was researched because it would allow testing of:

- Full login flow
- Shift acceptance flow
- Availability submission flow
- Document upload flow
- Notification handling
- Navigation between screens
- Error handling
- Real UI behaviour

This would improve confidence in production builds and reduce manual testing effort in the future.

Additionally, implementing E2E testing could help future contributors identify issues earlier during development rather than discovering them during manual testing later in the release cycle.

Comparison Between Current Testing and Maestro

Feature	Jest + React Native Testing Library	Maestro
Tests components	Yes	Limited
Tests user workflow	Partial	Yes
Runs entire app	No	Yes
Tests navigation	Partial	Yes
Tests device behaviour	No	Yes
Easy to maintain	Moderate	Moderate to Difficult
Setup complexity	Lower	Higher
CI/CD Integration	Easier	More setup required
Best for	Unit and Integration tests	End-to-end testing

Research Findings

1. Large Application Surface Area

The Guard App contains many screens and workflows, including:

- Login
- Signup
- Home
- Shifts
- Documents
- Availability

- Notifications
- Profile
- Settings
- Release Notes

Each major workflow would require its own Maestro flow files and testing scenarios.

Example:

- Valid login
- Invalid login
- Empty fields
- OTP flow
- Network failure
- Session expiration
- Navigation after login

This means that implementing Maestro properly would involve creating and maintaining a large number of test flows.

2. Time Required for Setup

Research showed that Maestro setup itself is relatively fast.

However, creating stable and reusable E2E tests is significantly more time consuming because:

- Element selectors need to be reliable
- Screens need consistent identifiers
- Async loading behaviour must be handled
- Authentication flows need setup handling
- Notifications and permissions may need mocking
- Different device states must be considered

Additionally, flows would need ongoing maintenance whenever UI changes are made.

3. Team Requirements

Based on the project size, implementing Maestro for the whole Guard App would realistically require:

- Approximately 2-3 contributors
- Around 2-4 weeks of consistent work
- Ongoing maintenance after implementation

This estimate includes:

- Initial setup
- Research
- Writing flows
- Debugging flaky tests
- Device configuration
- Documentation
- CI/CD experimentation

Because the trimester is already in later stages, completing this within the current timeline would not be practical.

4. Device and Environment Complexity

Maestro works directly with mobile devices or emulators.

This introduces additional complexity such as:

- Android emulator management
- Device permissions
- Build variants
- Expo compatibility considerations
- Notification handling
- Environment configuration

The Guard App also uses Expo and several native-related packages, which would require additional validation to ensure consistent Maestro behaviour.

Proposed Future Architecture

If Maestro were implemented in a future trimester or cohort, the following structure could be used:

```
.guard_app
├── maestro/
│   ├── login-flow.yaml
│   ├── shifts-flow.yaml
│   ├── availability-flow.yaml
│   ├── documents-flow.yaml
│   └── profile-flow.yaml
├── __tests__/
│   └── Login.test.js
├── src/
│   └── screens/
└── package.json
```

This would separate:

- Unit/integration tests
- End-to-end test flows
- Manual testing resources

Example Maestro Flows for SecureShift

Login flow

```
appId: com.secureshiftguardapp
---
- launchApp
- tapOn: "Enter your email"
- inputText: "guard@test.com"
- tapOn: "Enter your password"
- inputText: "password123"
- tapOn: "Login"
- assertVisible: "Home"
```

Invalid Login Flow

```
appId: com.secureshiftguardapp
---
- launchApp
- tapOn: "Login"
- assertVisible: "Please enter a valid email address"
```


Navigation Flow

```
appId: com.secureshiftguardapp
---
- launchApp
- tapOn: "Profile"
- assertVisible: "Edit Profile"
```

Benefits of Future Maestro Implementation

If implemented later, Maestro could provide several benefits:

1. Improved Regression Testing - Automated flows would help ensure that updates do not break existing features.
2. Reduced Manual Testing - Repeated flows could be automated instead of manually retesting every release.
3. Better Production Confidence - The team would gain more confidence before deployments and releases.
4. Faster QA Process - Critical workflows could be validated automatically.
5. Easier Future Expansion - Once the framework is established, future contributors could continue adding tests.

Limitations and Risks

Flaky Tests

E2E tests are more sensitive to:

- Timing issues
- UI changes
- Loading states
- Device performance

Maintenance Overhead

Tests require updates whenever UI or workflows change.

Additional Build Complexity

Running Maestro consistently across environments may require:

- CI/CD configuration
- Dedicated testing builds
- Stable emulator/device environments

Time Constraints

The largest limitation for the current trimester is available time.

A rushed implementation could result in:

- Unstable tests
- Poor maintainability
- Incomplete coverage
- More debugging than actual development value

Current Recommendation

For the current trimester, the recommended approach is:

Continue Using Jest + React Native Testing Library

The current testing stack already provides:

- Component testing
- Validation testing
- Navigation testing
- Render testing

This setup is currently more feasible within the remaining timeframe.

Prepare Documentation for Future Cohorts

Instead of fully implementing Maestro now, preparing documentation and research provides value for:

- Future students
- Future project contributors
- Future testing expansion

This allows future implementation to begin with existing research instead of starting from zero.

Suggested Future Implementation Plan

Phase 1 - Research and Setup

- Install Maestro
- Configure emulator/device setup
- Validate Expo compatibility
- Create proof-of-concept flow

Estimated Time:

- 2-4 days

Phase 2 - Core Authentication Flows

Implement:

- Login flow
- Invalid login flow
- Signup flow
- Logout flow

Estimated Time:

- 4-6 days

Phase 3 - Main Application Flows

Implement:

- Shift workflows
- Availability workflows
- Document workflows
- Notification workflows

Estimated Time:

- 1-2 weeks

Phase 4 - CI/CD and Maintenance

- Automated test execution
- Stable test environment setup
- Documentation updates
- Maintenance procedures

Estimated Time:

- Ongoing

Conclusion

Research into Maestro showed that it is a strong option for future end-to-end testing in the SecureShift Guard App.

It would provide better automated coverage for complete user workflows and could significantly improve long-term testing quality.

However, due to:

- Project size
- Existing workload
- Time limitations
- Required maintenance
- Environment complexity

A complete implementation is not currently feasible within the remaining trimester timeline.

The recommended approach is to continue improving the current Jest testing setup while preparing research and implementation guidance for future cohorts or future development phases.

This document can serve as a starting point for future contributors who may continue E2E testing implementation using Maestro.